



Dipartimento di Elettronica, Informazione e Bioingegneria

Politecnico di Milano

Prof. Matteo Camilli, Elisabetta Di Nitto, and Matteo Rossi

20133 Milano (Italia)

Piazza Leonardo da Vinci, 32

Tel. (39) 02-2399.3400

Fax (39) 02-2399.3411

Software Engineering 2 – Written Exam 1 (WE1)

July 19th, 2024

Last name, first name and Id number (matricola):

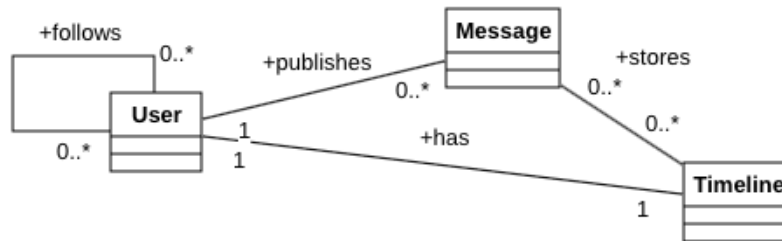
Number of paper sheets you are submitting as part of the exam:

Notes

- A. Write your name and Id number (matricola) on each piece of paper that you hand in.
- B. You may use a pencil.
- C. Unreadable handwriting is equivalent to not providing an answer.
- D. The use of any electronic devices (computer, smartphone, camera, etc.) is strictly forbidden, except for an ebook reader or a plain calculator.
- E. The exam is composed of three exercises. Read carefully all points in the text.
- F. Total available time for WE1: 1h and 30 mins**

Question 1 – Alloy (7 points)

Consider a social network like X where users can *follow* other existing users to see messages posted by them. The *following* relation is asymmetric, meaning that if *A* follows *B*, *B* does not necessarily follow *A*. Users can publish new messages to their followers. Users find messages published by the people they follow through their own *home timelines*. The following class diagram models the relationships between users, messages and timeline as they are defined in the description above.



Alloy_1 (3 point)

Define an Alloy model that captures the classes and relationships defined in the class diagram.

Alloy_2 (1 point)

Write a fact to ensure that users' timelines contain only messages published by other users they follow.

Alloy_3 (2 points)

If we want to represent the possibility for users to publish new messages for their followers, which parts of the Alloy model should become mutable and in which part of the model should temporal operators be used? Motivate your answer and explain how the model should be modified.

Alloy_4 (1 point)

Define a fact to ensure that all messages published by users are stored in the timelines of all their followers.

Solution

Alloy_1

```
sig User {
  follows: set User,
  has: Timeline
}{this not in follows}

fact {
  all t: Timeline | one u: User | u.has = t
}

sig Timeline {
  stores: set Message
}

sig Message {
  issuedBy: User
}
```

Alloy_2

```
fact {
  all t: Timeline | all m: Message | m in t.stores implies
    m.issuedBy in (has.t).follows
}
```

Alloy_3

We should consider as variable the Message signature together with its issueBy relation with User as this way we can have the possibility in our model to have new atom of type Message to be generated over the time. Clearly, also the binary relation issuedBy will have to be variable to accommodate the newly generated Message-User pairs. Similarly, the stores relation must be variable to model the fact that new messages can be inserted in all timelines.

Another element to be modified in the fact defined in point Alloy_2 which must be always true. Therefore, the model is modified as follows.

```
sig User {
  follows: set User,
  has: Timeline
}{this not in follows}

fact {
  all t: Timeline | one u: User | u.has = t
}

sig Timeline {
  var stores: set Message
}

var sig Message {
  var issuedBy: User
}

fact {
  always (all t: Timeline | all m: Message | m in t.stores implies m.issuedBy in
    (has.t).follows)
}

Alloy_4
fact {
  always (all m: Message, u: User | m.issuedBy in u.follows implies m in
    u.has.stores)
}
```

Question 2 – Software architectures (5 points)

Consider a system that allows users to launch and execute very time-consuming tasks, which sometimes return a result after a long time (many hours, or tens of hours), even when executed on powerful hardware. Typical examples of such systems are those that allow users to:

- launch very precise simulations of complex systems;
- execute optimization models to find the optimal solution;
- verify properties of formal models (for example, run predicates or execute assertions on Alloy models using very large values for bounds and/or steps).

We want to architect such a system in a way that allows users to perform other tasks while the time-consuming computation(s) are running.

In practice, the system allows users to launch tasks (which have been previously defined) and collect the results of each launched task after it finishes. However, users should be able to perform other operations (including launching other tasks) while the previously launched tasks are executed.

1. Define a suitable architecture (components, interfaces, deployment strategy) for systems with the above characteristics. Explain the rationale for your choices.
2. Describe through a UML sequence diagram the interaction that occurs among components when the user launches a new task, from task launch until the task completes its execution and the result is shown to the user.

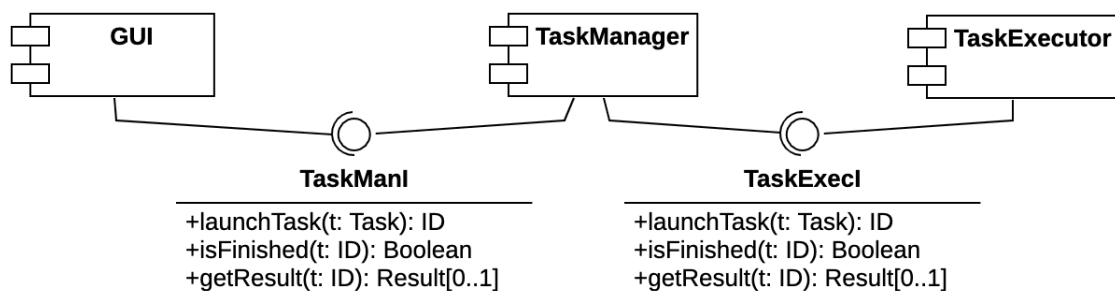
Assume that the results of a task are stored in a file, which is created when the task terminates.

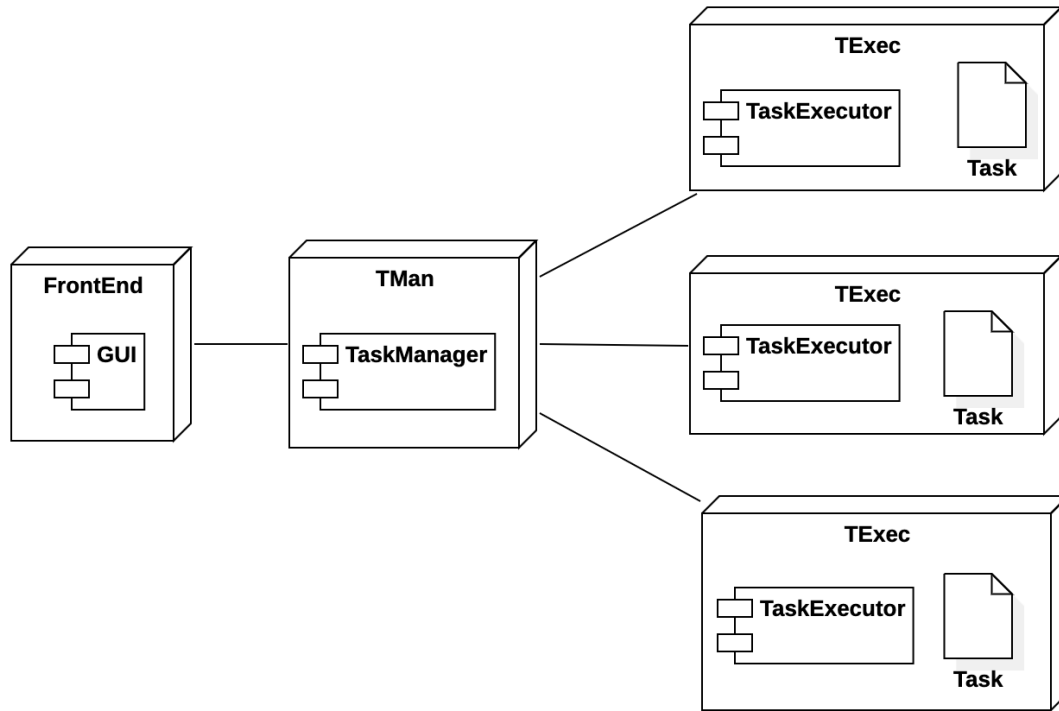
Solution

1.
The architecture should be designed so that execution of the time-consuming tasks is performed in an asynchronous manner with respect to other operations. More precisely, the system, after launching the task, should have a mechanism to let the user know when the task has finished, and collect the produced results only after the task has finished. This can be done in 2 typical ways: 1) a “pull” manner, in which the client/user checks if the task has finished and collects the results after it has determined that the task is over; or 2) a “push” manner, where the user is actively informed when the task has finished, and only at that point the user collects the results.

The following architecture adopts a “pull” approach: it is based on 2 components: one (*TaskManager*) for launching the tasks, and one (*TaskExecutor*) for executing (and monitoring) them. The roles of *TaskManager* and of *TaskExecutor* have been split to allow the system to have multiple *TaskExecutors* (where each *TaskExecutor* could be deployed on a separate node, so they do not interfere with one another, as shown in the Deployment Diagram below). If it is assumed that not many tasks will be executed in parallel, the two roles could be conflated together in a single component.

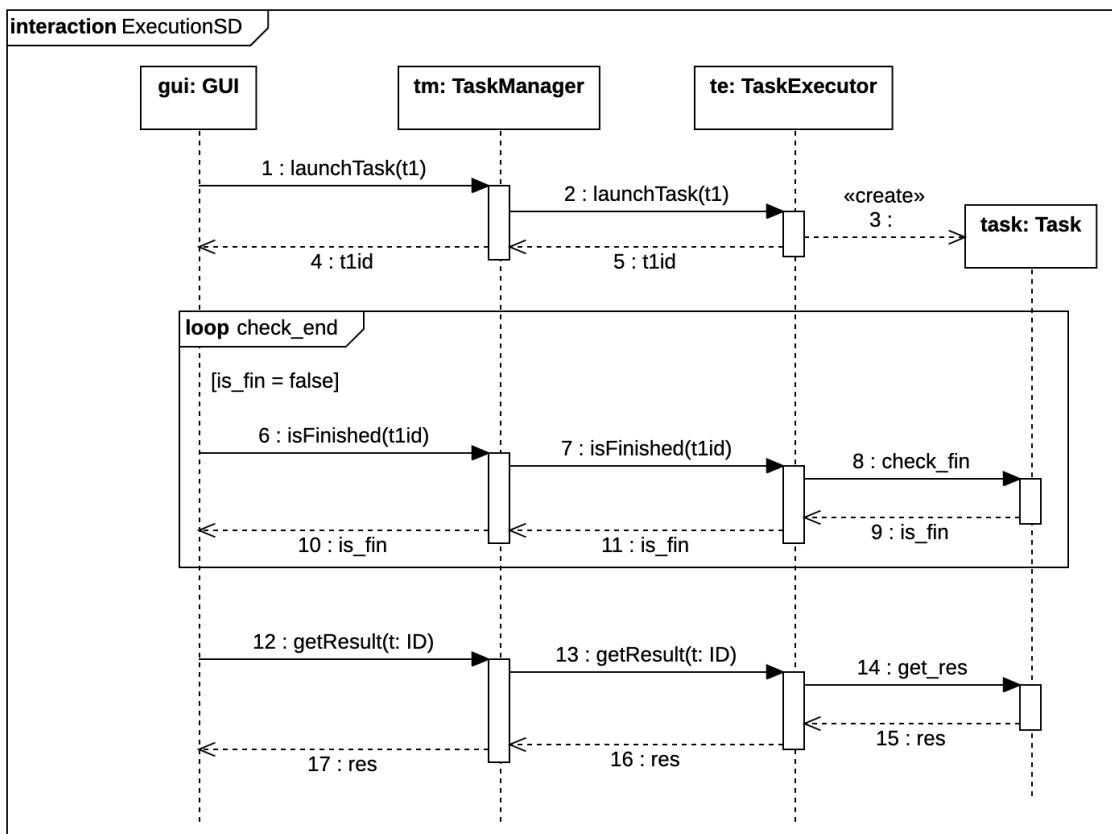
Notice that *TaskManager* (and also *TaskExecutor*) offer a primitive, *isFinished*, whose only role is to check whether the task has completed its execution.





2.

In the following diagram, the messages exchanged between the *TaskExecutor* and the *Task* are meant to depict not so much a direct interaction between the two, but the monitoring that *TaskExecutor* performs of the status of the task. This monitoring could be as simple as checking that a suitable output file has been produced by the task.



Question 3 – Concolic execution (4 points)

Consider the following program fragment.

```
1. void foo(int a, int b) {
2.   if (a < b) {
3.     a = 4;
4.   } else {
5.     b = 4;
6.   }
7.   b = b + 3;
8.   a = qux(a, b);
9.   if (a > 10) {
10.    b = b - 2;
11.  }
12. }
```

In the next points, assume that `qux(a, b)` returns `a+b`.

Concolic_1 (1 point)

Run a concolic execution with seed inputs $a = 1$ and $b = 2$ and write down the path condition at the end of `foo`.

Concolic_2 (3 points)

Starting from the first execution, proceed using the concolic approach to explore all possible paths in the program. For each explored path provide the symbolic and actual values assumed by all variables and the corresponding path condition.

Solution

Concolic_1

Line	Concrete Execution	Symbolic Execution	Path condition
1	$a = 1, b = 2$	$a = A, b = B$	N/A
2,3,4,6	$a = 4, b = 2$	$a = 4, b = B$	$A < B$
7	$a = 4, b = 5$	$a = 4, b = B + 3$	$A < B$
8	$a = 9, b = 5$	$a = \text{qux}(4, B+3), b = B + 3$	$A < B$
9	$a = 9, b = 5$	$a = \text{qux}(4, B+3), b = B + 3$	$A < B \wedge \text{qux}(4, B+3) \leq 10$
11, 12	$a = 9, b = 5$	$a = \text{qux}(4, B+3), b = B + 3$	$A < B \wedge \text{qux}(4, B+3) \leq 10$

Concolic_2

We negate the last part of the path condition to explore the path 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, thus: $A < B \wedge \text{qux}(4, B+3) > 10$. We cannot solve it symbolically. We look for a value for B that fulfils the new condition. For instance: $B = 4$. We obtain the following execution:

Line	Concrete Execution	Symbolic Execution	Path condition
1	$a = 1, b = 4$	$a = A, b = B$	N/A
2,3,4,6	$a = 4, b = 4$	$a = 4, b = B$	$A < B$
7	$a = 4, b = 7$	$a = 4, b = B + 3$	$A < B$
8	$a = 11, b = 7$	$a = \text{qux}(4, B+3), b = B + 3$	$A < B$
9	$a = 11, b = 7$	$a = \text{qux}(4, B+3), b = B + 3$	$A < B \wedge \text{qux}(4, B+3) > 10$
10	$a = 9, b = 5$	$a = \text{qux}(4, B+3), b = B + 1$	$A < B \wedge \text{qux}(4, B+3) > 10$
11,12	$a = 9, b = 5$	$a = \text{qux}(4, B+3), b = B + 1$	$A < B \wedge \text{qux}(4, B+3) > 10$

Now we proceed negating the first part of the path condition. In this case, we do not care about the second part. We just want to execute the else branch (line 5). We can draw the following values $A=4$ and $B=1$ so that $A \geq B$ holds. We obtain the following execution:

Line	Concrete Execution	Symbolic Execution	Path condition
1	$a = 4, b = 1$	$a = A, b = B$	N/A
2	$a = 4, b = 1$	$a = A, b = B$	$A \geq B$
4, 5, 6	$a = 4, b = 4$	$a = A, b = 4$	$A \geq B$
7	$a = 4, b = 7$	$a = A, b = 7$	$A \geq B$
8	$a = 11, b = 7$	$a = \text{qux}(A, 7), b = 7$	$A \geq B$
9	$a = 11, b = 7$	$a = \text{qux}(A, 7), b = 7$	$A \geq B \wedge \text{qux}(A, 7) > 10$
10	$a = 11, b = 5$	$a = \text{qux}(A, 7), b = 5$	$A \geq B \wedge \text{qux}(A, 7) > 10$
11,12	$a = 11, b = 5$	$a = \text{qux}(A, 7), b = 5$	$A \geq B \wedge \text{qux}(A, 7) > 10$

Once again, we negate the last part of the path condition to explore the last path: $A \geq B \wedge \text{qux}(A, 7) \leq 10$. The following values fulfil the path condition: $A = 3, B = 3$. In this case we obtain the following execution:

Line	Concrete Execution	Symbolic Execution	Path condition
1	$a = 3, b = 3$	$a = A, b = B$	N/A
2	$a = 3, b = 3$	$a = A, b = B$	$A \geq B$
4, 5, 6	$a = 3, b = 4$	$a = A, b = 4$	$A \geq B$
7	$a = 3, b = 7$	$a = A, b = 7$	$A \geq B$
8	$a = 10, b = 7$	$a = \text{qux}(A, 7), b = 7$	$A \geq B$
9	$a = 10, b = 7$	$a = \text{qux}(A, 7), b = 7$	$A \geq B \wedge \text{qux}(A, 7) \leq 10$
11,12	$a = 10, b = 7$	$a = \text{qux}(A, 7), b = 7$	$A \geq B \wedge \text{qux}(A, 7) \leq 10$

In conclusion, the following table summarizes the paths we followed, the test cases we have derived and the resulting path condition.

Path	Test case	Path condition
1, 2, 3, 4, 6, 7, 8, 9, 11, 12	$a = 1, b = 2$	$A < B \wedge \text{qux}(4, B+3) \leq 10$
1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12	$a = 1, b = 4$	$A < B \wedge \text{qux}(4, B+3) > 10$
1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12	$a = 4, b = 1$	$A \geq B \wedge \text{qux}(A, 7) > 10$
1, 2, 4, 5, 6, 7, 8, 9, 11, 12	$a = 3, b = 3$	$A \geq B \wedge \text{qux}(A, 7) \leq 10$